

AI-Based Intelligent Dead Reckoning — Research Brief

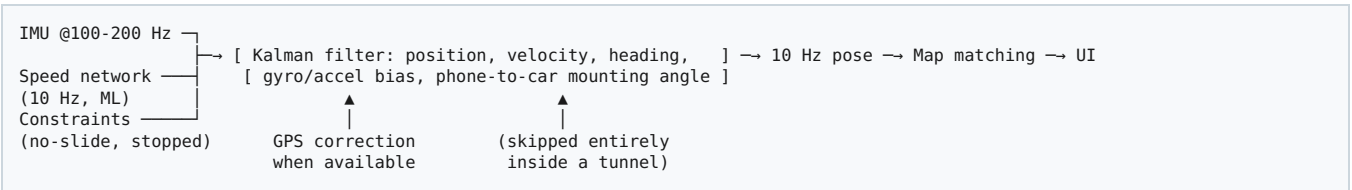
Smart India Hackathon 2026 · Problem Statement 26168 · Condensed engineering survey

1. The short answer

Build **one continuously-running Kalman filter** that always uses the phone's IMU to predict where the car is, and simply *switches off* the GPS correction step when GPS disappears. Feed it three things a neural network cannot get wrong quietly:

- 1. A network that **predicts the car's forward speed** directly from the IMU (never integrate acceleration twice — that explodes).
- 2. **Physics constraints**: a car cannot slide sideways or fly, and when it stops, velocity and rotation are exactly zero.
- 3. **Map matching**: snap the estimated path onto the OpenStreetMap road graph, which kills most of the sideways error.

The single biggest insight: because there is only one filter and GPS is just an optional correction, the "seamless transition within milliseconds" requirement is satisfied *by design*, not by clever code. Teams that build two separate systems and hand off between them will show a visible position jump on every tunnel entry and lose marks.



2. Vocabulary you'll see everywhere in this field

Term	Plain meaning
Dead reckoning (DR)	Estimating where you are now from where you were, plus how fast and in what direction you've been moving. No external reference.
Strapdown INS	The naive version: integrate gyroscope to get heading, integrate accelerometer twice to get position. Fails within seconds on a phone.
EKF / Error-state EKF	Extended Kalman Filter. Tracks a best estimate <i>and</i> how uncertain it is. "Error-state" means it tracks the small correction to the estimate rather than the estimate itself — numerically much better for navigation.
InEKF	Invariant EKF. Same idea, but the state (rotation, velocity, position) is treated as living on a Lie group. The linearisation stops depending on your current estimate, so it converges even from a badly wrong start — exactly the "phone mounted at a random angle" case.
NHC	Non-Holonomic Constraint. A fake measurement you feed the filter saying "sideways velocity = 0, vertical velocity = 0", because cars don't crab or fly. Free accuracy.
ZUPT / ZARU	Zero-velocity / zero-angular-rate update. When you detect the car is stopped (traffic light), tell the filter velocity = 0 and rotation rate = 0. This is how you measure and cancel gyro bias.
Bias	A constant offset a MEMS sensor reports even when nothing is happening. Gyro bias is the enemy (see §5).
Map matching	Snapping a noisy trajectory onto a road network. The standard method is a Hidden Markov Model (Newson & Krumm, 2009) solved with Viterbi.
ATE / drift %	Absolute Trajectory Error (metres) and drift as a percentage of distance travelled. PS 26168 grades you on the latter: under 10%.

3. Why the obvious approach fails

A phone accelerometer has a bias of roughly $10\text{ mg} \approx 0.098\text{ m/s}^2$. Position error from integrating that twice is:

$$e = \frac{1}{2} \cdot b_a \cdot t^2 \rightarrow \frac{1}{2} \times 0.098 \times 60^2 \approx \textbf{176 m after just 60 seconds}$$

That alone blows the 100 m budget for a 1 km tunnel. So: **never integrate acceleration to get speed**. Regress speed with a network, or constrain it. Every serious paper in this area does this.

4. The six blocks the problem statement demands

Required capability	What to actually build
Alignment & calibration (phone pitch/roll/yaw vs car)	Pitch and roll come free from the gravity vector (IO-VNBD even gives you gravity channels). Yaw comes from running PCA on horizontal acceleration during hard braking / acceleration — the principal axis is the car's forward direction. Better: put the mounting angle <i>into the filter state</i> and let it converge under NHC. Add a "phone got bumped" detector that re-inflates its uncertainty.

AI speed & vibration filter	A 1D CNN / TCN / small ResNet over a ~1–2 s IMU window, regressing forward speed and its uncertainty . Predicting uncertainty matters as much as predicting speed — the filter needs to know how much to trust it.
Map matching + kinematics	Online HMM map matching over an offline OSM graph. Key adaptation: instead of GPS noise, use your filter's own covariance as the emission probability. Commit road segments older than ~5–10 s, keep recent ones tentative.
GNSS+INS fusion	The same filter. The "AI" part is a small network that adjusts the filter's noise settings on the fly (relax the no-sideways-slide rule when cornering hard, tighten it on straights). This is exactly Brossard's AI-IMU and it is only ~6,200 parameters.
Seamless handler	Gate the GPS update with a statistical (chi-squared) test. No mode switch exists, so latency is zero. On tunnel exit, reject the first few multipath-corrupted fixes and run a short backward smoother so the drawn path corrects smoothly instead of snapping.
Navigation UI	Vector map, smooth 10 Hz car icon, mode indicator, and a visible uncertainty ellipse . Judges consistently respond to seeing the covariance grow inside the tunnel and collapse on exit.

5. Attack heading first — the error budget

Speed error is bounded (the network caps it) and map matching corrects along-track error anyway. **Heading error is unbounded and pushes you sideways**, where nothing can save you — if the heading is wrong enough, map matching confidently snaps you onto the wrong road.

For a car going straight at speed `v` with constant gyro bias `b_g`, heading error grows linearly and lateral position error grows quadratically:

$$\Delta\psi(t) = b_g \cdot t \quad e_{lateral} \approx \frac{1}{2} \cdot b_g \cdot v \cdot t^2$$

Worked example (the PS benchmark case): 1 km tunnel at 60 km/h ($v = 16.7$ m/s, $t = 60$ s), budget 100 m. Solving gives `b_g` < 0.19 $^{\circ}$ /s. That looks easy — but that is the budget for the *entire* heading error, and a phone gyro that hasn't been bias-calibrated in that drive can sit well above it. Add scale-factor error during turns and bias random walk, and it gets tight fast.

Practical consequence: estimate and cancel gyro bias aggressively during every GPS-available second, snapshot it at tunnel entry, and use `TYPE_GYROSCOPE_UNCALIBRATED` on Android so you get the raw reading and the OS's bias estimate *separately* — otherwise the OS silently subtracts a bias your filter is also trying to estimate, and the two fight.

Sensor grades, for context

Parameter	Phone MEMS	Tactical MEMS	FOG (the edge engine target)
Gyro bias instability	~10-100 $^{\circ}$ /hr	~0.3-3 $^{\circ}$ /hr	<0.01-0.1 $^{\circ}$ /hr
Angular random walk	~0.5-5 $^{\circ}$ /vhr	~0.1-0.3 $^{\circ}$ /vhr	<0.05 $^{\circ}$ /vhr
Accel bias instability	~0.1-1 mg	~10-50 μ g	~1-10 μ g
Scale factor error	~1-5 %	~100-1000 ppm	~10-100 ppm

To get your own numbers: log the phone stationary for several hours at max rate, compute the overlapping **Allan deviation** $\sigma(\tau)$. The $-1/2$ slope at $\tau = 1$ s gives random walk; the flat minimum gives bias instability. Use these to set the filter's process noise instead of guessing.

6. Which published models are actually worth your time

Method	What it does	Reported result	Domain	Use it?
AI-IMU Brossard 2020	Small CNN adjusts the noise covariance of NHC pseudo-measurements inside an Invariant EKF. IMU only.	1.10% translational error on KITTI , beating LiDAR/stereo methods	Car	YES — copy this architecture. Code: github.com/mbrossar/ai-imu-dr
Onyekpe INS Appl.Sci. 2021	RNN/LSTM/GRU corrects INS displacement + orientation-rate error.	Up to 89.6% / 93.4% improvement over raw INS on IO-VNBD	Car, phone-grade	YES — this is your official baseline on the mandated dataset
TLIO 2020	Small ResNet regresses 3D displacement <i>and its covariance</i> , fused tightly into an EKF.	33% less position drift, 27% less yaw drift vs RoNIN	Pedestrian	Framework yes, weights no — retrain on car data
EqNIO ICLR 2025	Makes the network mathematically invariant to how the device is rotated about gravity.	New SOTA inertial-only; better on unseen orientations	Pedestrian	Idea yes — this is the principled fix for "phone mounted at any angle"
AirIMU / AirIO 2023-25	Learns IMU correction plus preintegration uncertainty; AirIO shows body-frame input beats world-frame.	AirIO: +66.7% from body-frame representation	Vehicle / drone	Yes — take the body-frame lesson and the uncertainty head
WhONet / R-WhONet	RNN (only ~8.2k params) correcting <i>wheel-speed</i> odometry.	180 s outage: 3.93 m error; ~0.15% of distance	Car, wheel	NOT a legal solution (needs OBD) — quote only as the ceiling
VeTorch / Cortés-Solin	CNN/TCN regresses vehicle forward speed from phone IMU, fed to an EKF as a constraint.	Speed from phone IMU alone, validated in-vehicle	Car / handheld	Yes — this is your speed block
RoNIN, IONet, RIDI, IDOL, CTIN, iMoT	Pedestrian inertial odometry — regress 2D velocity or displacement from IMU.	ATE of a few metres, indoors, walking	Pedestrian	No. They learn a walking-gait prior (0.5-2 m/s). A car at 16.7 m/s is off-distribution and they under-predict badly.

Don't quote these numbers as your own promise. AI-IMU's 1.10% was measured on KITTI's automotive-grade IMU, and WhONet's sub-1% needs wheel speed. Neither is a phone-only result. Your honest reference point is the Onyekpe INS numbers.

7. IO-VNBD — the mandated dataset

Onyekpe et al., *Data in Brief* 35:106885 (2021). Repo: github.com/onyekpeu/IO-VNBD. Roughly 5,700 km across UK, France and Nigeria. Two parallel streams:

Stream	Rate	Contents	Use it?
"V-" (car ECU)	10 Hz	29 columns via CAN + VBOX GPS: wheel speeds, steering angle, yaw rate, brake pressure, engine rpm, GPS lat/lon/velocity/heading	Wheel channels are off-limits; GPS here is useful as ground truth
"S-" (smartphone)	10 Hz (GPS 1 Hz)	24 columns via AndroSensor: accel XYZ, gravity XYZ , gyro yaw/pitch/roll, magnetometer XYZ, orientation, GPS lat/lon/speed/accuracy	This is your training data

Frame and ground truth: navigation frame is North-East-Down; tracking is 2D (yaw-only rotation); true displacement between GPS fixes computed with Vincenty's inverse geodesic formula; VBOX GPS accuracy $\sim \pm 3$ m. There are dedicated stationary segments (>20 min) for bias estimation — use them.

Baseline hyperparameters to reproduce

Setting	Value (from the Onyekpe / WhONet papers)
Window length	1 second (10 samples at 10 Hz)
Loss	MAE
Optimizer / LR	Adamax, $7e-4$
Batch / dropout	128 / 0.05
Model size	Vanilla RNN, 1 hidden layer, 72 units ($\sim 8,200$ params)
Outage simulation	Non-overlapping synthetic outages of 10 / 30 / 60 / 120 / 180 s on held-out scenarios
Metrics	CTE = Cumulative True Error (signed sum), CRSE = Cumulative Root Square Error. These are <i>not</i> cross-track error or ATE — a common misreading.

Three traps in this dataset

1. **Phone data is only 10 Hz.** You cannot demonstrate a 200 Hz pipeline on it. Use "S-" for the app story and say explicitly in the deck which stream produced which plot.
2. **Wheel speed is present but banned.** Easy to leak into your feature set by accident. Audit your dataloader.
3. **No Indian roads.** Collect a few hours yourself — Delhi/NCR has plenty of underpasses, expressway tunnels and basement parking. A domain-shift ablation (train on IO-VNBD, test on your own data) is something almost no other team will submit.

8. Map matching, practically

The Hidden Markov Model formulation treats each candidate road segment as a hidden state. **Emission probability** = how far the estimate is from that road (Gaussian). **Transition probability** = penalises the mismatch between straight-line distance and actual on-road route distance between consecutive points. Viterbi finds the most likely road sequence.

Libraries, ranked for your use case:

- **GraphHopper map-matching** (Java, uses `hmm-lib`, imports OSM directly, Android-friendly) — most practical for the app.
- **FMM / fast-map-matching** (C++/Python, precomputed lookup table, extremely fast) — best if you want the matcher inside your C++ core.
- **Valhalla Meili** (C++, tiled, mobile-oriented); **BMW Barefoot** (Java, has both offline and online HMM variants).

Offline graph: take a Geofabrik `.pbf` extract, filter to drivable `highway=*`, build a compact adjacency structure. City-scale is tens of MB — fine on a phone. Prune to a bounding box for the demo.

Where map matching cannot help: a multi-level car park has no road graph. Switch to a separate mode: tight NHC dead reckoning plus **barometer floor detection** (one parking level $\approx 2.5\text{--}3$ m $\approx 0.3\text{--}0.36$ hPa step) and ramp detection from sustained pitch. Be honest in the submission that this is the hardest case and drift may exceed 10% on a long stay.

9. Android and deployment realities

- **200 Hz cap.** Since Android 12 (API 31), `registerListener()` is limited to 200 Hz unless you declare `HIGH_SAMPLING_RATE_SENSORS`. `SensorDirectChannel` is capped near 50 Hz without it. iOS CoreMotion typically tops out near 100 Hz.
- **Background is dead.** Since Android 9, continuous sensors deliver no events in the background. You need a **foreground service** with a persistent notification.
- **Use uncalibrated sensor types** so you get raw values and the OS bias estimate in separate fields (see the warning in §5).
- **Timestamps.** Sensor timestamps are a monotonic device clock in nanoseconds, with jitter and batching; GPS is UTC. Convert both to one base and interpolate — never assume a nominal Δt .

- **Threading:** filter propagation on a high-rate thread reading a lock-free ring buffer; network inference at 1-10 Hz on a second thread posting measurement updates; GPS callback on a third.
- **Inference stack:** LiteRT/TFLite (best Android delegates) or ExecuTorch (PyTorch-native, matches your stack). **FP16 is safe; INT8 is risky for the covariance head** — a mis-scaled covariance corrupts a Kalman filter worse than a slightly noisy mean. Test that specifically.
- **One core, two frontends:** write the filter in C++ or Rust, expose to Android via JNI, and reuse the identical library as the FOG-grade edge engine at 200 Hz. This satisfies the "edge deployable engine" requirement by construction instead of as a second project — and it plays directly to your embedded background.

10. Is <10% drift actually achievable?

Configuration	Result	Verdict
Raw phone strapdown INS	~176 m in 60 s	Hopeless alone
Learned INS correction (Onyekpe, phone-grade)	8.62 m max error over ~220 m on the hard acceleration scenario, vs 79 m raw	Honest phone-grade reference point
AI-IMU: IEKF + NHC + learned covariance	1.10% on KITTI	Proves the constraint approach works — but on a better IMU
Wheel odometry (WhONet)	~0.15% of distance	Ceiling you can't reach without OBD

Verdict: yes for tunnels and underpasses (tens of seconds up to ~1–2 minutes), *provided* you combine learned speed + NHC + ZUPT/ZARU + a continuously running filter + map matching. Without map matching you will occasionally miss 10% on curved tunnels and roundabouts. Underground car parks are the genuinely hard case.

Dominant error terms, in order: (1) heading drift, (2) speed estimate error, (3) phone-to-car mounting angle error, (4) gyro scale factor during turns.

11. India-specific notes

- **NavIC is unreliable through standard Android APIs.** Chipsets that support it include Snapdragon 720G/765G/662/460/8 Gen 1, Google Tensor G2, Apple A17 Pro. But a 2026 study in *Remote Sensing Letters* found NavIC signals are often filtered out by low-level drivers even where hardware support exists, and Android's GNSS API lacks standardised NavIC identifiers. **Treat NavIC as a bonus; rely on GPS + GLONASS + Galileo + BeiDou.**
- **OSM in India** is good for major roads in metros, weaker on service roads and flyover-vs-below tagging. Expect flyover/service-road confusion to be a real map-matching failure mode in Delhi.
- **Extra datasets worth pretraining on:** comma2k19 (33 h highway, phone-grade IMU + CAN — the best supplement), KITTI (better IMU, used by AI-IMU), Google Smartphone Decimeter Challenge (real Android raw GNSS + IMU in cars). Skip EuRoC/TUM-VI — drone and handheld, no GNSS.

12. Build order (what to do first)

1. **Reproduce the Onyekpe baseline on IO-VNBD's smartphone channels.** 1 s windows, MAE, Adamax 7e-4, synthetic 10/30/60/180 s outages. You need this number to beat, and the screening round explicitly asks for position plots from an IO-VNBD subset.
2. **Stand up the error-state InEKF** in C++/Rust with NHC + ZUPT + ZARU, no learning yet. If tuning alone can't get within ~3–5× of the GPS-available baseline over 60 s outages, fix the constraints before adding model complexity.
3. **Add the learned speed head** (TCN or 1D-ResNet, predicting speed and variance). Train on IO-VNBD "S-" plus comma2k19.
4. **Add AI-IMU's learned adaptive NHC covariance.** It's ~6k parameters — cheap, and it's the mechanism with the strongest published car result.
5. **Add mounting-angle estimation** (in-filter, or EqNIO-style canonicalisation) plus a bump detector.
6. **Add online HMM map matching** over an offline OSM graph, driven by filter covariance.
7. **Wrap in the Android app** last. The core is the deliverable; the app is the demo.

Given your embedded C, FPGA and PyTorch background, the sensible split is that you own the **C++/Rust filter core and the edge engine** — that's the half the sponsor actually cares about, given the FOG-IMU and EW-resilience framing in the problem statement — and hand the Android UI to someone else on the team.

13. Caveats to keep you honest

- Pedestrian ATE figures (RoNIN, TLIO) are not comparable to vehicle drift percentages. They rank architectures; they don't predict car performance.
- The WhONet paper is internally inconsistent on outage-sequence counts (809/399/197/129 in the intro vs 688/342/168/111 in the tables) and reports a suspicious identical CTE for both methods at 120 s. Verify against the tables before you cite it.
- Several 2025–26 architectures (FTIN, IONext, GNIO, MosaicIMU, SNN+InEKF) are recent preprints with little independent replication. Treat their headline numbers as provisional.
- The under-10% target in a multi-level underground car park is the least certain claim in this whole brief. Say so in the submission rather than letting a judge find it.